

# UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Minería de Datos

Semestre I – 2026



*Excelencia que trasciende*

**DEL VALLE**  
GRUPO EDUCATIVO

## **Laboratorio 10**

### **Aprendizaje semi-supervisado**

#### **Integrantes:**

Nájera, Nina – 231088

Ciprian, Mishell – 231169

Ramírez, Diego – 23601

# Índice

|                                                |           |
|------------------------------------------------|-----------|
| <b>1. Introducción</b>                         | <b>2</b>  |
| <b>2. Repositorio y fuente de datos</b>        | <b>2</b>  |
| <b>3. Descripción del dataset</b>              | <b>2</b>  |
| <b>4. Análisis exploratorio</b>                | <b>3</b>  |
| <b>5. Preprocesamiento</b>                     | <b>7</b>  |
| <b>6. Diseño experimental semi-supervisado</b> | <b>8</b>  |
| <b>7. Modelos implementados</b>                | <b>8</b>  |
| <b>8. Resultados</b>                           | <b>8</b>  |
| <b>9. Matrices de confusión</b>                | <b>10</b> |
| <b>10. Visualización con PCA</b>               | <b>13</b> |
| <b>11. Discusión</b>                           | <b>15</b> |
| <b>12. Conclusiones</b>                        | <b>15</b> |

## 1. Introducción

En este laboratorio se trabajó con aprendizaje semi-supervisado usando el dataset *NASA access log dataset 1995*. Este conjunto de datos contiene registros de solicitudes HTTP realizadas al servidor web del Kennedy Space Center de NASA durante 1995.

El objetivo principal fue comparar un modelo supervisado tradicional contra dos modelos semi-supervisados. Para esto, se simuló un escenario donde solo una parte pequeña de los datos tenía etiqueta disponible y el resto se trató como no etiquetado.

El problema se planteó como una tarea de clasificación binaria. Las solicitudes con código de respuesta 200 fueron clasificadas como exitosas. Las solicitudes con otro código de respuesta fueron clasificadas como no exitosas. Esta división permite evaluar si los modelos pueden aprender patrones del tráfico web aunque tengan pocas etiquetas al inicio.

Los modelos evaluados fueron Random Forest supervisado, Self-Training con Random Forest y Label Spreading. Los porcentajes de datos etiquetados evaluados fueron 5 %, 10 % y 20 %.

## 2. Repositorio y fuente de datos

El repositorio utilizado para el laboratorio se encuentra en el siguiente enlace:

<https://github.com/Ninaswiftie09/Mineria-de-Datos/tree/Laboratorio10>

El dataset utilizado fue *NASA access log dataset 1995*, disponible en Kaggle:

<https://www.kaggle.com/datasets/souhagaa/nasa-access-log-dataset-1995?resource=download>

La fuente original de estos registros corresponde al archivo NASA-HTTP del Internet Traffic Archive. Estos logs representan solicitudes HTTP al servidor web del Kennedy Space Center de NASA. Cada registro contiene información como el host, fecha y hora, solicitud realizada, código de respuesta y bytes devueltos por el servidor.

## 3. Descripción del dataset

El dataset contiene información de tráfico web. Cada fila representa una solicitud realizada al servidor. A partir de estos registros se analizaron variables relacionadas con el método HTTP, la URL solicitada, el código de respuesta, el tamaño de la respuesta y datos derivados del tiempo y del texto de la solicitud.

La Tabla 1 muestra las variables principales utilizadas en el análisis.

Tabla 1: Variables principales del dataset

| Variable | Descripción                                             |
|----------|---------------------------------------------------------|
| host     | Equipo o dirección que realizó la solicitud al servidor |
| time     | Fecha y hora en que se realizó la solicitud             |
| method   | Método HTTP utilizado en la solicitud                   |
| url      | Recurso solicitado al servidor                          |
| response | Código de respuesta HTTP devuelto por el servidor       |
| bytes    | Cantidad de bytes devueltos en la respuesta             |

Para mejorar el análisis y el desempeño de los modelos, se generaron variables derivadas a partir de los campos originales. Estas variables ayudan a representar mejor el comportamiento de cada solicitud.

Tabla 2: Variables derivadas usadas para el modelado

| Variable      | Descripción                                             |
|---------------|---------------------------------------------------------|
| bytes_log     | Transformación logarítmica de los bytes                 |
| hour          | Hora de la solicitud                                    |
| dayofweek     | Día de la semana de la solicitud                        |
| url_length    | Longitud del texto de la URL                            |
| url_depth     | Profundidad de la URL según la cantidad de niveles      |
| has_query     | Indica si la URL contiene una consulta                  |
| host_length   | Longitud del texto del host                             |
| extension_top | Tipo de recurso solicitado según la extensión principal |
| target        | Clase objetivo usada para clasificación                 |

## 4. Análisis exploratorio

La variable objetivo fue creada a partir del código de respuesta HTTP. La clase exitosa representa solicitudes con respuesta 200. La clase no exitosa representa solicitudes con códigos diferentes a 200.

La distribución de clases muestra que la mayoría de solicitudes fueron exitosas. Esto indica que el dataset está desbalanceado y que por eso es importante revisar métricas adicionales al accuracy.

Tabla 3: Distribución de clases

| Clase      | Cantidad | Porcentaje |
|------------|----------|------------|
| exitosa    | 2696600  | 90.93 %    |
| no_exitosa | 268961   | 9.07 %     |

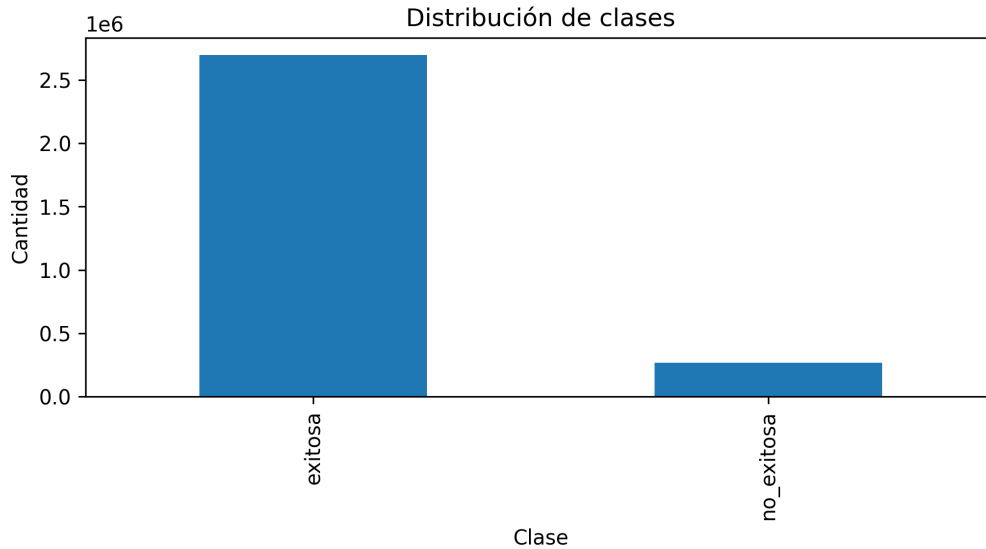


Gráfico 1: Distribución de clases

El Gráfico 1 muestra que la mayoría de solicitudes pertenecen a la clase exitosa. Esto confirma que el dataset está desbalanceado, ya que las solicitudes no exitosas representan una parte mucho menor del total. Por esta razón, no basta con revisar solo accuracy, sino que también es necesario analizar métricas como F1-score y recall.

También se revisó la distribución de códigos de respuesta. El código 200 fue el más frecuente. Los códigos 302, 304, 404 y 501 aparecen con menor frecuencia. Esto confirma que el tráfico está dominado por solicitudes exitosas.

Tabla 4: Distribución de códigos de respuesta

| Código de respuesta | Cantidad |
|---------------------|----------|
| 200                 | 2696600  |
| 302                 | 6912     |
| 304                 | 258161   |
| 404                 | 3858     |
| 501                 | 30       |

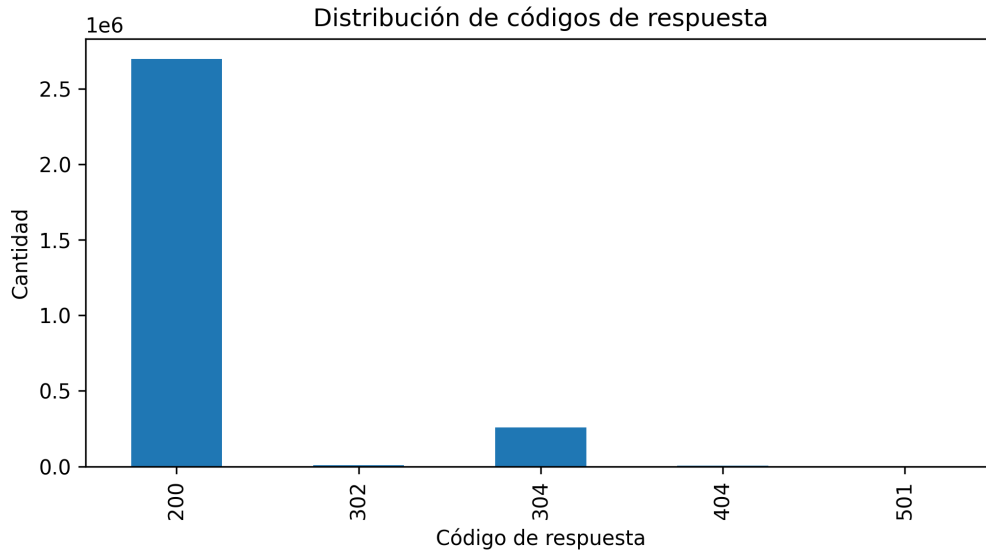


Gráfico 2: Distribución de códigos de respuesta

El Gráfico 2 permite observar que el código 200 es el más frecuente dentro del dataset. Esto significa que la mayoría de solicitudes fueron procesadas correctamente por el servidor. Los demás códigos aparecen con menor frecuencia, lo cual explica por qué la clase no exitosa tiene menos registros.

En la distribución de métodos HTTP se observa que el método GET domina casi todo el tráfico. Los métodos HEAD y POST aparecen en menor cantidad. Esto es esperable en un servidor web, ya que la mayoría de solicitudes corresponden a consultas de recursos.

Tabla 5: Distribución de métodos HTTP

| Método | Cantidad |
|--------|----------|
| GET    | 2957882  |
| HEAD   | 7649     |
| POST   | 30       |

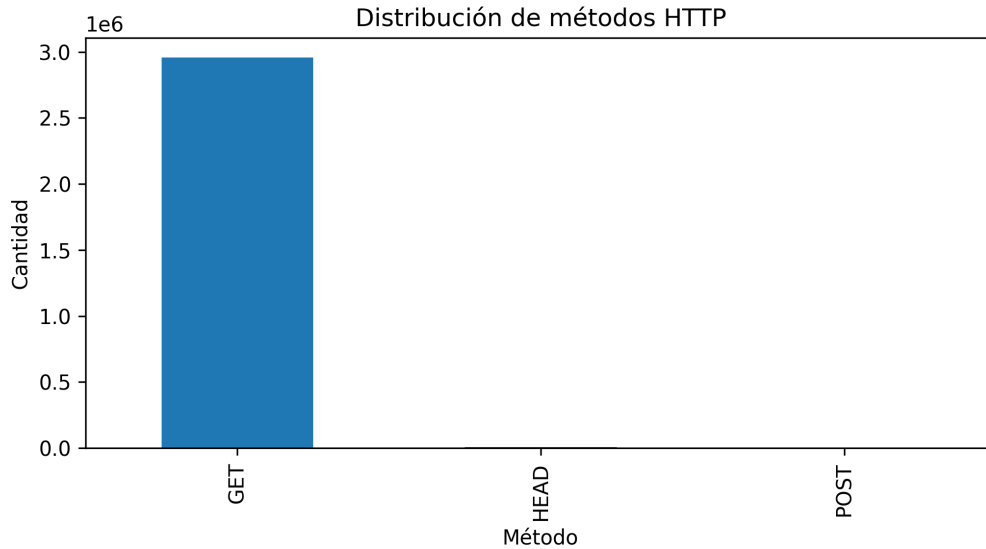


Gráfico 3: Distribución de métodos HTTP

El Gráfico 3 muestra que el método GET domina casi todo el tráfico del servidor. Esto es normal en registros de acceso web, ya que la mayoría de usuarios solicitan páginas, imágenes u otros recursos. Los métodos HEAD y POST aparecen en cantidades mucho menores.

El análisis de bytes transformados muestra una diferencia clara entre las solicitudes exitosas y no exitosas. Las solicitudes exitosas tienden a tener mayor cantidad de bytes, mientras que las no exitosas se concentran cerca de valores bajos. Esta diferencia ayuda a que los modelos puedan separar ambas clases con buen desempeño.

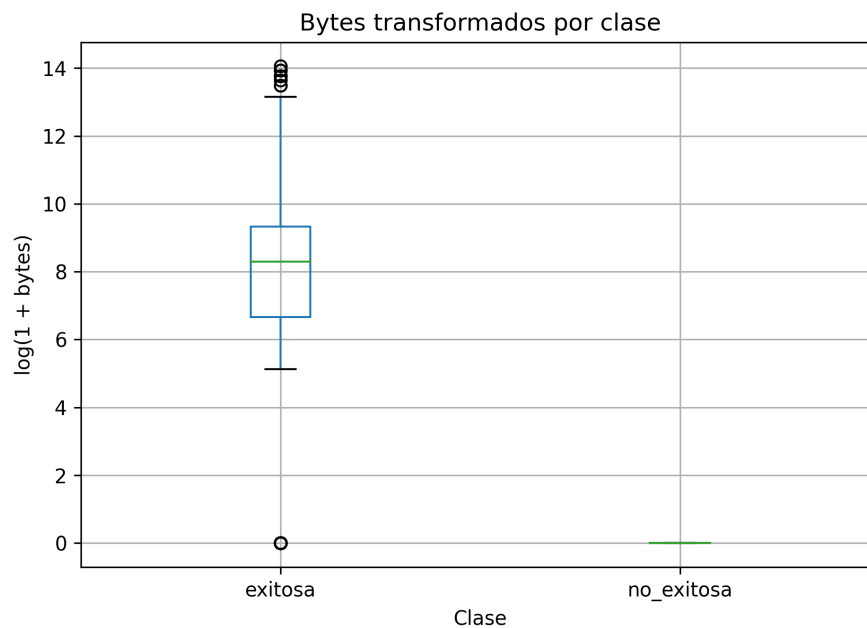


Gráfico 4: Bytes transformados por clase

El Gráfico 4 muestra una diferencia clara entre las solicitudes exitosas y no exitosas. Las

solicitudes exitosas presentan valores más altos de bytes transformados, mientras que las no exitosas se concentran cerca de cero. Esta separación ayuda a explicar por qué los modelos obtuvieron un desempeño alto.

La matriz de correlación permite observar relaciones entre variables numéricas. Se observa una relación alta entre `url_length` y `url_depth`, ya que una URL más larga suele tener más niveles. También se observa una relación esperada entre `bytes` y `bytes_log`.

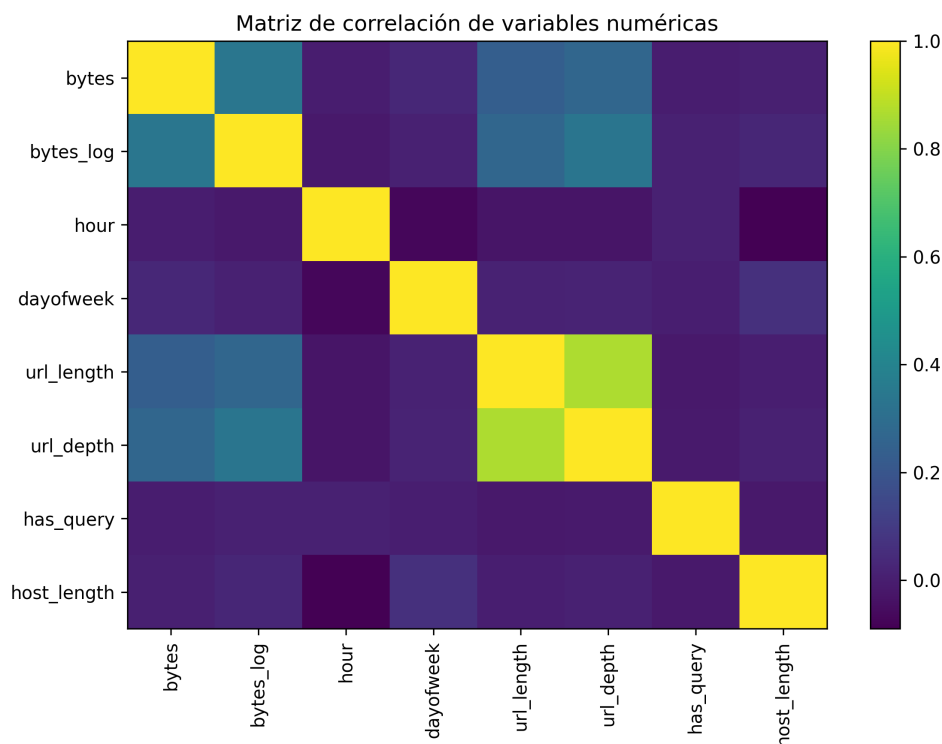


Gráfico 5: Matriz de correlación de variables numéricas

El Gráfico 5 muestra la relación entre las variables numéricas usadas en el análisis. Se observa una relación fuerte entre la longitud de la URL y la profundidad de la URL, ya que una URL con más niveles normalmente también es más larga. También se observa una relación esperada entre `bytes` y `bytes transformados`.

## 5. Preprocesamiento

Primero se limpiaron los datos y se revisaron valores nulos. Luego se construyó la variable objetivo usando el código de respuesta HTTP.

Después se generaron variables derivadas para representar mejor la información del tráfico web. Se usaron transformaciones sobre la URL, el host, el tiempo y los bytes. La variable `bytes_log` se creó para reducir el efecto de valores muy grandes en la variable de bytes.

Las variables numéricas fueron estandarizadas para que tuvieran una escala comparable. Las variables categóricas fueron codificadas con One Hot Encoding. Este paso fue necesario porque los modelos utilizados trabajan con datos numéricos.



Para reducir el tiempo de ejecución y mantener el balance de clases, se tomó una muestra estratificada de los datos. Luego se separó el conjunto en entrenamiento y prueba.

Tabla 6: Separación de datos

| Conjunto      | Cantidad de datos | Uso                        |
|---------------|-------------------|----------------------------|
| Entrenamiento | 4500              | Entrenar los modelos       |
| Prueba        | 1500              | Evaluar el desempeño final |

## 6. Diseño experimental semi-supervisado

El diseño experimental buscó simular una situación donde solo una parte pequeña del conjunto de entrenamiento tiene etiquetas. El resto de datos se mantuvo como no etiquetado para que los algoritmos semi-supervisados aprovecharan su estructura.

Se evaluaron tres porcentajes de etiquetas: 5 %, 10 % y 20 %. En cada caso se comparó un modelo supervisado contra dos modelos semi-supervisados.

Tabla 7: Escenarios con datos etiquetados y no etiquetados

| Porcentaje etiquetado | Datos etiquetados | Datos no etiquetados |
|-----------------------|-------------------|----------------------|
| 5 %                   | 225               | 4275                 |
| 10 %                  | 450               | 4050                 |
| 20 %                  | 900               | 3600                 |

## 7. Modelos implementados

El primer modelo fue Random Forest supervisado. Este modelo se usó como referencia para comparar el desempeño de los métodos semi-supervisados. Se entrenó solo con la parte etiquetada del conjunto de entrenamiento.

El segundo modelo fue Self-Training con Random Forest. Este método entrena un clasificador inicial con los datos etiquetados. Luego usa ese clasificador para asignar pseudo-etiquetas a los datos no etiquetados cuando la predicción tiene suficiente confianza. En este experimento se probaron thresholds de 0.75, 0.85 y 0.95.

El tercer modelo fue Label Spreading. Este método propaga etiquetas usando la estructura de cercanía entre los datos. La idea es que observaciones similares deberían tener etiquetas similares. En este experimento se usó kernel kNN y se probaron 5, 10 y 15 vecinos.

## 8. Resultados

La Tabla 8 muestra las mejores configuraciones encontradas para cada modelo. Los tres modelos alcanzaron resultados altos. Esto indica que las variables utilizadas permiten separar bien las solicitudes exitosas de las no exitosas.

Tabla 8: Mejores resultados por modelo

| Modelo           | Etiquetas | Hiperparámetro   | Accuracy | F1 ponderado |
|------------------|-----------|------------------|----------|--------------|
| Label Spreading  | 20 %      | n_neighbors=15   | 0.996    | 0.996        |
| Self-Training RF | 10 %      | threshold=0.95   | 0.996    | 0.996        |
| Supervisado RF   | 5 %       | n_estimators=120 | 0.996    | 0.996        |

En el Gráfico 6 se observa que Random Forest supervisado y Self-Training mantuvieron un desempeño estable con los diferentes porcentajes de etiquetas. Label Spreading tuvo menor desempeño con 5 % de etiquetas, pero mejoró cuando se aumentó la cantidad de datos etiquetados.

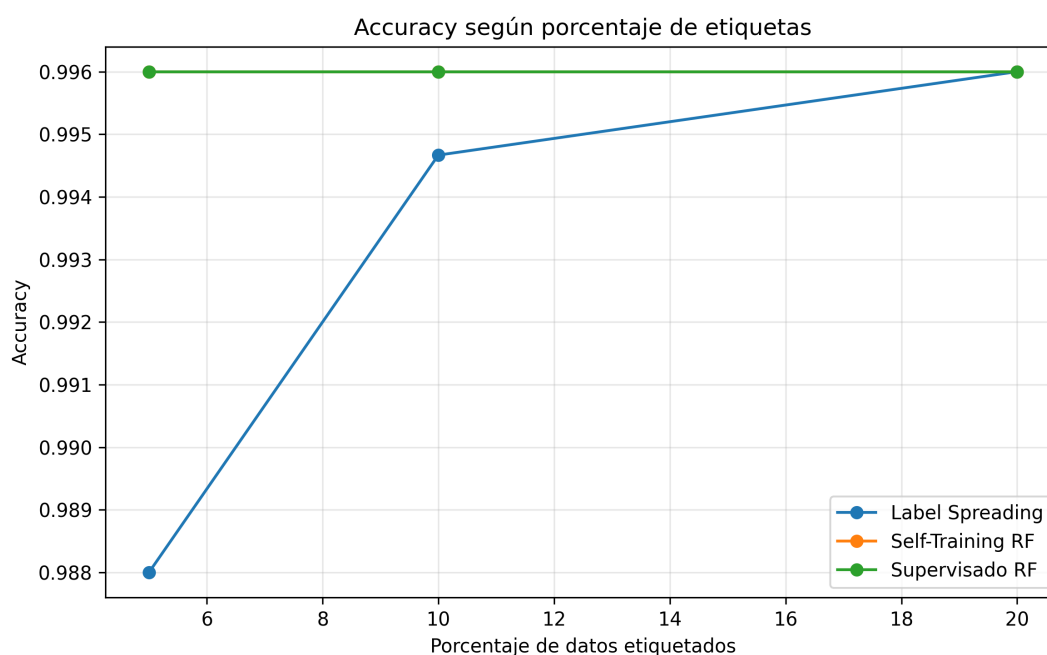


Gráfico 6: Accuracy según porcentaje de etiquetas

El Gráfico 6 compara el accuracy de los modelos usando 5 %, 10 % y 20 % de datos etiquetados. Random Forest supervisado y Self-Training RF mantienen un desempeño estable. Label Spreading empieza con menor accuracy cuando solo se usa 5 % de etiquetas, pero mejora al aumentar la cantidad de datos etiquetados.

El Gráfico 7 muestra el F1-score ponderado. Esta métrica es importante porque el dataset está desbalanceado. Aunque el accuracy fue alto, el F1-score ayuda a revisar si el modelo también está funcionando bien para la clase minoritaria.

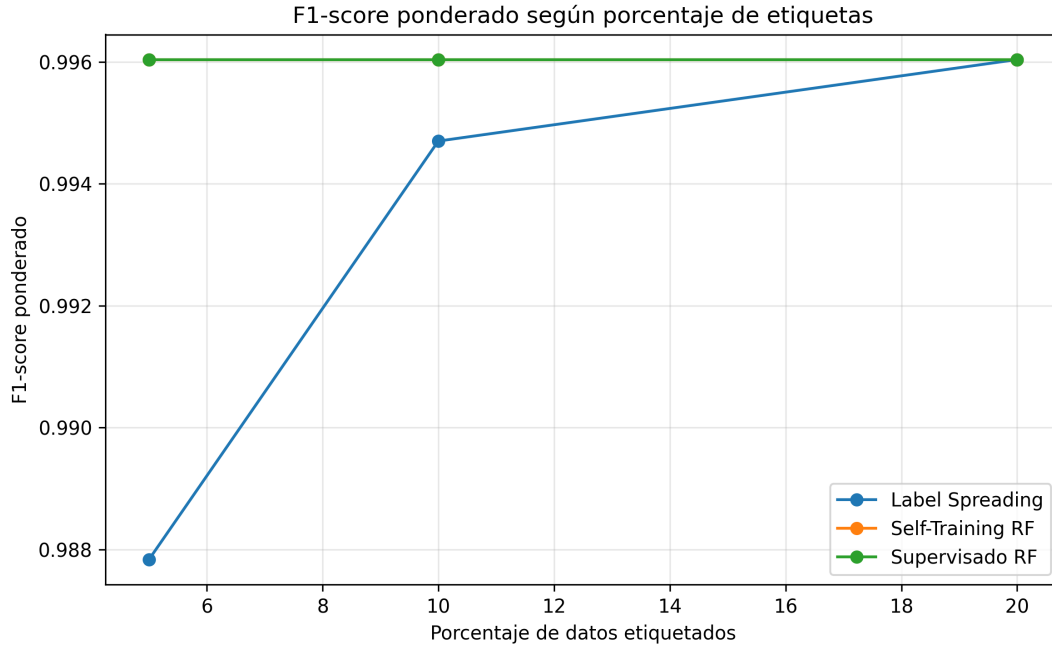


Gráfico 7: F1-score ponderado según porcentaje de etiquetas

El Gráfico 7 muestra el F1-score ponderado para cada modelo. Esta métrica es útil porque considera el desempeño general tomando en cuenta el desbalance de clases. El comportamiento es similar al accuracy, ya que Label Spreading mejora cuando recibe más etiquetas, mientras que Random Forest supervisado y Self-Training RF se mantienen estables.

La Tabla 9 muestra el reporte de clasificación del mejor modelo. Se observa que el modelo tuvo buen desempeño en ambas clases. La clase no exitosa obtuvo recall de 1.000, lo que indica que todas las solicitudes no exitosas del conjunto de prueba fueron identificadas correctamente.

Tabla 9: Reporte de clasificación del mejor modelo

| Clase        | Precision | Recall | F1-score | Soporte |
|--------------|-----------|--------|----------|---------|
| exitosa      | 1.000     | 0.996  | 0.998    | 1364    |
| no_exitosa   | 0.958     | 1.000  | 0.978    | 136     |
| macro avg    | 0.979     | 0.998  | 0.988    | 1500    |
| weighted avg | 0.996     | 0.996  | 0.996    | 1500    |

## 9. Matrices de confusión

Las matrices de confusión muestran que los tres mejores modelos tuvieron el mismo comportamiento en el conjunto de prueba. Los modelos clasificaron correctamente 1358 solicitudes exitosas y 136 solicitudes no exitosas. El único error observado fue que 6 solicitudes exitosas fueron clasificadas como no exitosas.

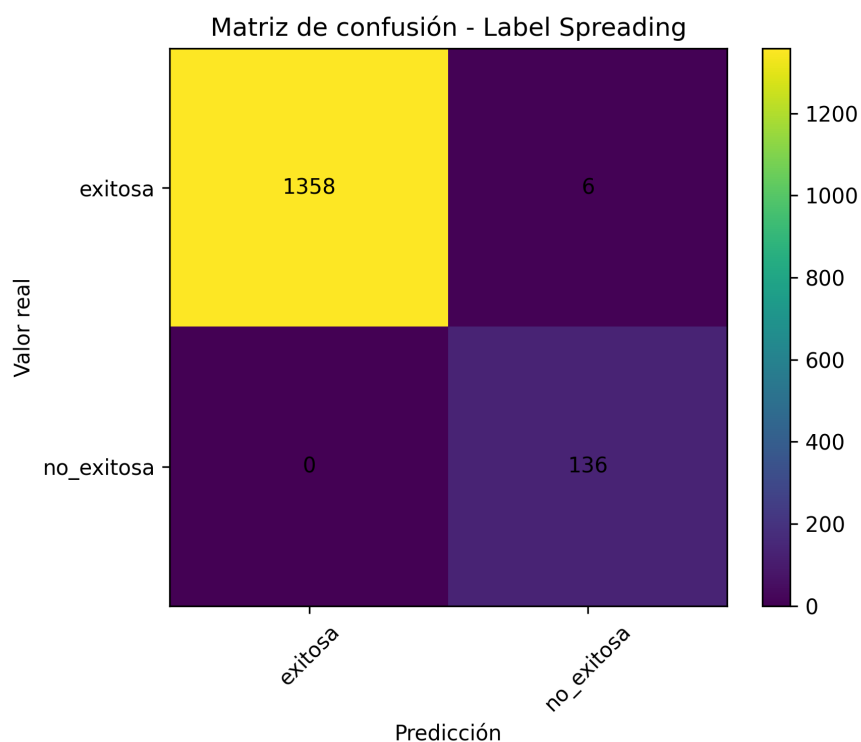


Gráfico 8: Matriz de confusión de Label Spreading

El Gráfico 8 muestra que Label Spreading clasificó correctamente todas las solicitudes no exitosas del conjunto de prueba. También clasificó correctamente la mayoría de solicitudes exitosas, con solo 6 errores en esa clase. Esto indica que el modelo logró capturar bien la diferencia entre ambas clases.

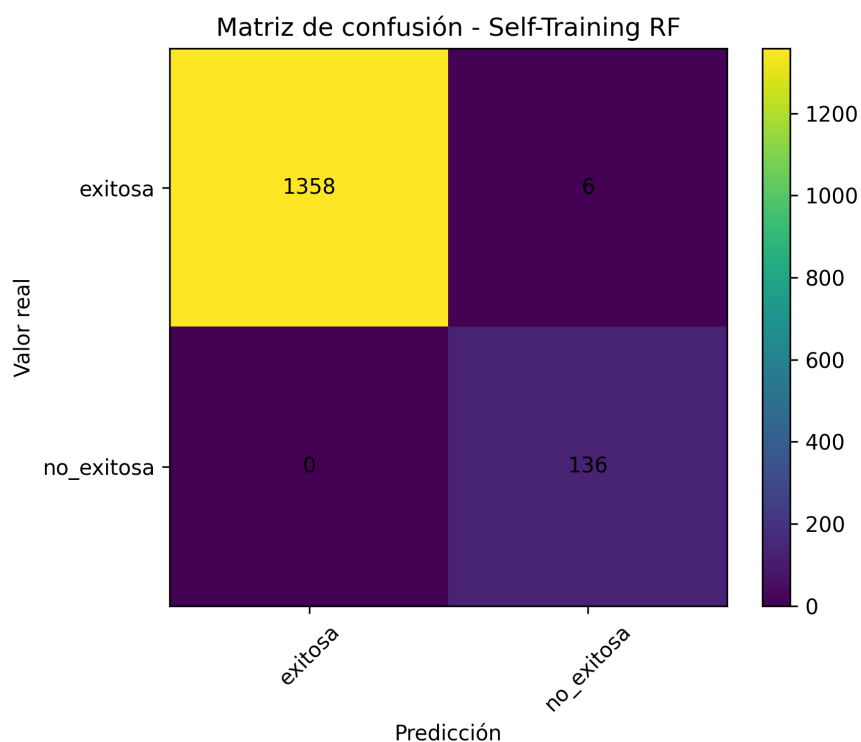


Gráfico 9: Matriz de confusión de Self-Training RF

El Gráfico 9 muestra que Self-Training RF obtuvo el mismo comportamiento que Label Spreading en la matriz de confusión. El modelo identificó correctamente todas las solicitudes no exitosas y solo confundió 6 solicitudes exitosas. Esto demuestra que el pseudo-etiquetado funcionó bien en este conjunto de datos.

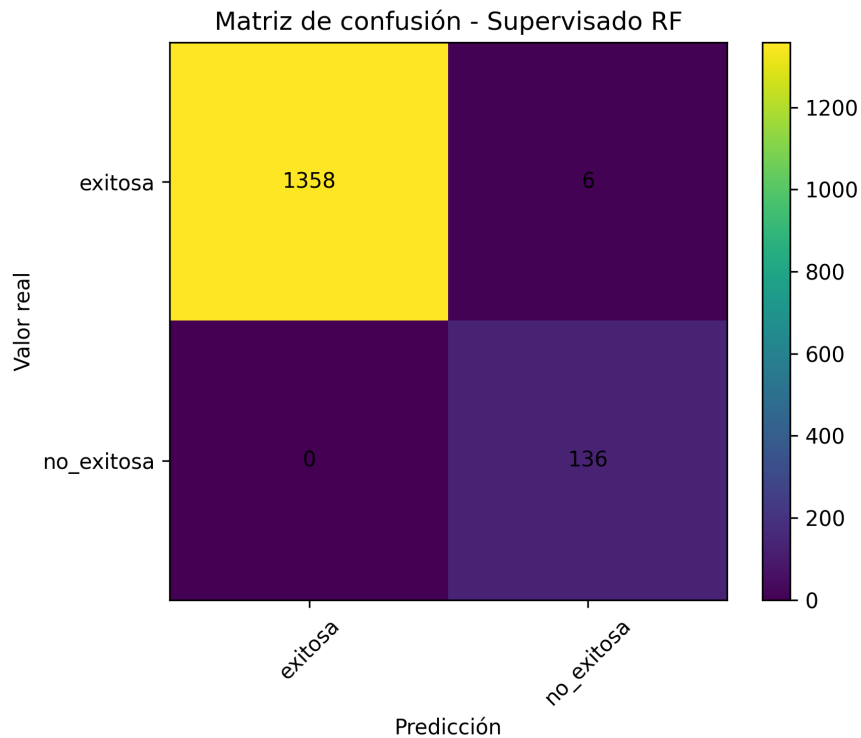


Gráfico 10: Matriz de confusión de Random Forest supervisado

El Gráfico 10 muestra que Random Forest supervisado también tuvo un desempeño alto. A pesar de usar una cantidad reducida de etiquetas, logró clasificar correctamente casi todos los registros del conjunto de prueba. Esto confirma que las variables usadas tienen suficiente información para separar las clases.

## 10. Visualización con PCA

Para visualizar el comportamiento de las clases se aplicó PCA y se redujeron los datos a dos componentes principales. El Gráfico 11 muestra los valores reales y el Gráfico 12 muestra las predicciones del mejor modelo.

Ambas gráficas son muy parecidas, lo que indica que el modelo logró reproducir bien la separación de las clases. Esto también confirma lo observado en las matrices de confusión.

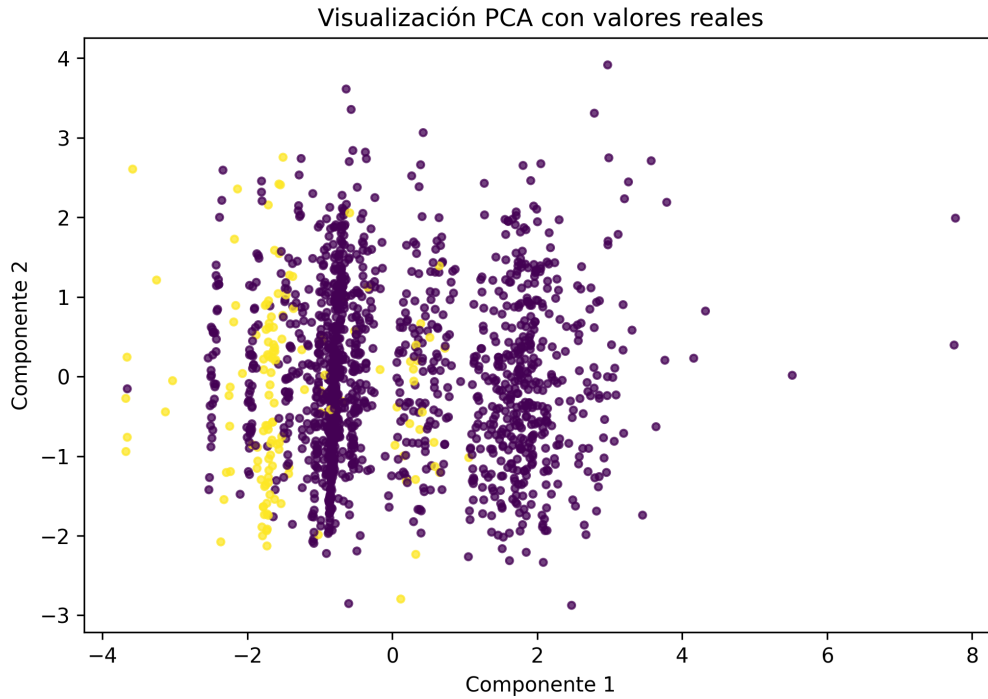


Gráfico 11: Visualización PCA con valores reales

El Gráfico 11 muestra los datos reales proyectados en dos componentes principales. Aunque las clases no quedan separadas de forma perfecta en dos dimensiones, sí se observa una tendencia donde los puntos de la clase no exitosa se agrupan en ciertas zonas. Esto ayuda a visualizar la estructura general del problema.

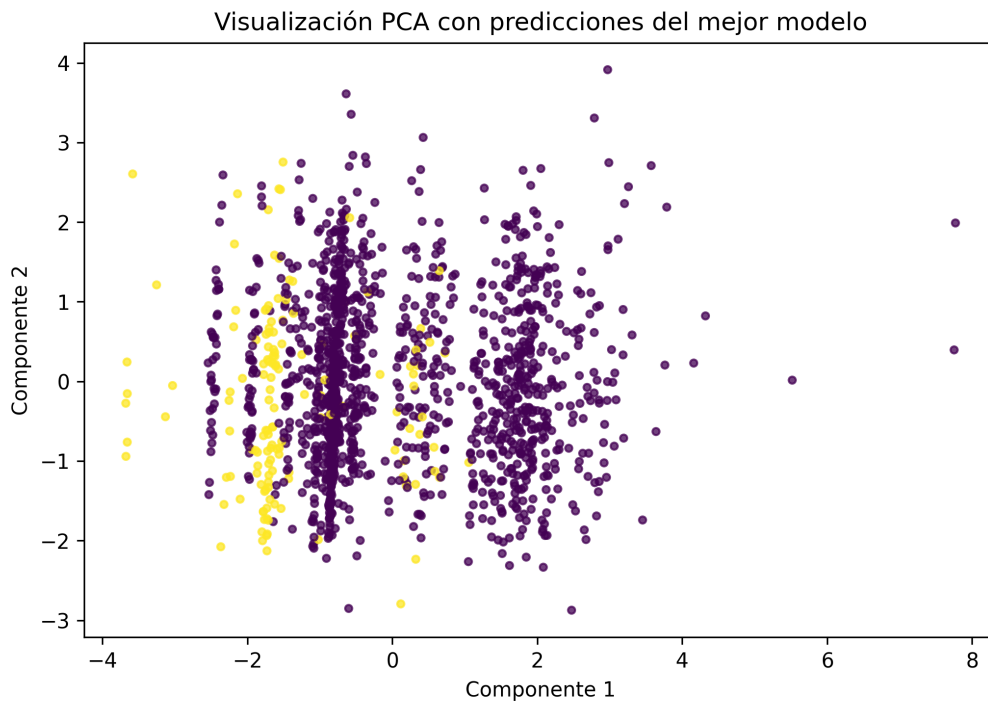


Gráfico 12: Visualización PCA con predicciones del mejor modelo

El Gráfico 12 muestra las predicciones del mejor modelo usando la misma proyección PCA. La distribución es muy parecida a la de los valores reales, lo cual indica que el modelo reprodujo correctamente la separación principal entre clases. Esto coincide con los buenos resultados observados en las métricas y matrices de confusión.

## 11. Discusión

Los resultados muestran que los tres modelos lograron un desempeño alto. Esto se debe a que el dataset tiene variables que ayudan bastante a separar las clases. En especial, la cantidad de bytes tiene un comportamiento diferente entre solicitudes exitosas y no exitosas.

Random Forest supervisado logró buenos resultados incluso con una cantidad pequeña de etiquetas. Esto indica que el patrón principal del problema se puede aprender con pocos ejemplos. Self-Training también fue estable, ya que el modelo base generó pseudo-etiquetas confiables para los datos no etiquetados.

Label Spreading fue más sensible al porcentaje de etiquetas. Con 5 % de datos etiquetados tuvo menor desempeño, pero al aumentar la cantidad de etiquetas mejoró hasta alcanzar el mismo resultado que los otros modelos. Esto tiene sentido porque este algoritmo depende bastante de la estructura de cercanía entre los datos y de la calidad de las etiquetas iniciales.

No se observó subajuste, ya que los modelos lograron métricas altas. Tampoco se observó un problema claro de sobreajuste en los resultados de prueba. Aun así, el desempeño debe interpretarse con cuidado porque el dataset tiene una separación bastante clara entre las clases.

## 12. Conclusiones

El mejor modelo global fue Random Forest supervisado, ya que alcanzó un accuracy de 0.996 y un F1-score ponderado de 0.996 usando solo 5 % de datos etiquetados.

Entre los modelos semi-supervisados, Label Spreading con 20 % de datos etiquetados y 15 vecinos fue uno de los mejores resultados. Self-Training con Random Forest también alcanzó un desempeño equivalente usando un threshold de 0.95.

El aprendizaje semi-supervisado fue útil porque permitió aprovechar datos no etiquetados dentro del entrenamiento. En este caso, los métodos semi-supervisados lograron resultados competitivos frente al modelo supervisado. Esto muestra que este enfoque puede ser útil cuando se tienen muchos datos disponibles, pero pocas etiquetas.

La principal limitación del experimento es el desbalance de clases. Aunque las métricas fueron altas, siempre es necesario revisar F1-score, recall y matrices de confusión para evitar conclusiones basadas solo en accuracy.